Conocimiento, lógica e inferencia

Introducción

Piensa en un médico que tiene a un paciente con fiebre alta, tos y dificultad para respirar. En cuestión de segundos descarta enfermedades, recuerda casos parecidos, pondera riesgos y cruza información. Al final dice: 'probablemente, es neumonía bacteriana, se necesita un hemocultivo y una radiografía'. La pregunta que se va a responder a lo largo de esta lectura es si se puede hacer eso mismo con una computadora.

Para responderla, primero vale conectar con lo que ya se estudió. Los algoritmos de búsqueda son poderosos, pero tienen una particularidad y es que operan sin saber realmente qué están buscando. No entienden el dominio, no distinguen entre buscar una ruta en un mapa, resolver un *puzzle* o planificar el movimiento de un robot, sino que aplican la misma lógica de exploración en todos los casos. Eso funciona bien para ciertos problemas, pero en cuanto los problemas se vuelven más específicos y complejos, esa generalidad se convierte en una limitación.

> Un sistema que diagnostica enfermedades, que clasifica variantes genéticas o que detecta intrusiones en una red necesita algo más que explorar caminos. En definitiva, necesita representar conocimiento sobre el dominio y usarlo para razonar.

Eso es exactamente lo que aborda esta lectura y es también una de las preguntas más antiguas de la inteligencia artificial. La tradición que intenta responderla se llama IA simbólica, que se estudió en el módulo 1 y su herramienta principal es la lógica formal.

Russell y Norvig (2016) llaman agentes basados en conocimiento a los sistemas que mantienen una representación interna del mundo y razonan sobre ella para tomar decisiones. El componente clave es la base de conocimiento que, a diferencia de una base de datos, no solo devuelve lo que

fue explícitamente almacenado, sino que permite derivar información nueva a partir de lo que ya contiene. Para que eso funcione se necesitan tres cosas: un lenguaje para representar el conocimiento, un conjunto de reglas para manipularlo y una semántica que le dé significado.

A lo largo de la lectura se recorrerán los fundamentos de la jerarquía entre datos y conocimiento, la lógica proposicional y la de primer orden, y cómo se construyen sistemas basados en reglas. Son los cimientos sobre los que se apoyan los sistemas expertos, los lenguajes de consulta, las ontologías médicas y los planificadores robóticos, que se continuará desarrollando en la próxima lectura.

Caso: MYCIN y la formalización del conocimiento médico

En 1972, Edward Shortliffe inició en la Universidad de Stanford lo que se convertiría en uno de los proyectos más influyentes de la historia de la inteligencia artificial: el desarrollo de MYCIN, un sistema experto diseñado para diagnosticar infecciones bacterianas en la sangre y recomendar tratamientos con antibióticos (Shortliffe, 1976).

El problema que MYCIN intentaba resolver era muy concreto. Las infecciones bacterianas severas requieren tratamiento inmediato, pero identificar el microorganismo causante tarda entre 24 y 48 horas mediante cultivos de laboratorio. Durante ese tiempo, el médico debe decidir qué antibiótico administrar sin tener certezas. Esa decisión, tomada con información incompleta, puede marcar la diferencia entre la vida y la muerte del paciente, y depende de años de experiencia clínica que no todos los profesionales tienen en igual medida.

El equipo de Shortliffe (1976) pasó años entrevistando a especialistas en enfermedades infecciosas del Hospital Universitario de Stanford. El objetivo era extraer el conocimiento de la mente de los expertos y codificarlo en un formato que una computadora pudiera utilizar para razonar. El resultado fue una base de conocimiento de aproximadamente 600 reglas de producción. Una regla típica de MYCIN tenía esta estructura: "SI el organismo es gramnegativo Y el organismo es bacilar Y el paciente tiene infección del tracto urinario, ENTONCES hay evidencia (0,85) de que el organismo es Escherichia coli".

Ese número entre paréntesis, el factor de certeza, fue una de las innovaciones centrales del proyecto. MYCIN no trabajaba con verdades absolutas, sino con grados de confianza, lo que le permitía razonar en condiciones de información incompleta o contradictoria. Este mecanismo anticipa algo que se explorará con más profundidad en la próxima lectura, cuando se aborde el razonamiento bajo incertidumbre.

Los resultados de las evaluaciones fueron sorprendentes. En pruebas ciegas donde médicos expertos y el sistema evaluaban los mismos casos sin saber quién era quién, MYCIN alcanzó un nivel de desempeño comparable al de los especialistas, y claramente superior al de médicos residentes. En una evaluación publicada en 1979, el sistema obtuvo valoraciones positivas en el 65 % de los casos, frente al 42,5 % de los médicos sin especialidad en enfermedades infecciosas.

Sin embargo, MYCIN nunca se implementó en hospitales reales. Los obstáculos no fueron técnicos, sino institucionales, éticos y regulatorios: ¿quién es responsable si el sistema se equivoca? ¿Puede un software respaldar una prescripción? ¿Están los médicos obligados a seguir sus recomendaciones? Estas preguntas, que en 1979 carecían de respuesta, siguen siendo centrales en el debate sobre sistemas de IA en medicina hoy.

Lo que MYCIN dejó como legado fue doble. Por un lado, demostró que el conocimiento experto podía formalizarse y que un sistema podía razonar sobre él de manera útil. Por otro lado, expuso con claridad el que sería el talón de Aquiles de la IA simbólica durante décadas, precisamente la adquisición del conocimiento. Construir y mantener esa base de 600 reglas fue costoso en tiempo y recursos, y el conocimiento médico cambia con el tiempo. No era un proceso sostenible a escala.

A modo de reflexión

MYCIN podía razonar con 600 reglas codificadas a partir del conocimiento de expertos. Pero los especialistas que lo evaluaron observaron que, en ciertos casos, el sistema llegaba a la respuesta correcta por razones incorrectas y en otros descartaba diagnósticos

que un médico experimentado hubiera considerado por su intuición construida a lo largo de años de práctica clínica.

> ✓ ¿Qué necesita un sistema para razonar sobre conocimiento de dominio de manera útil, explicable y sostenible?

Anota tu respuesta, ya que se desarrollará este tema a lo largo de la lectura.

Conocimiento, inferencia y razonamiento

Hay una distinción clásica que vale explicar bien porque aparece constantemente en este campo y es la diferencia entre datos, información, conocimiento y sabiduría. Se la conoce como la jerarquía DIKW por sus siglas en inglés: *Data, Information, Knowledge, Wisdom*. Aunque parece sencilla, tiene implicaciones profundas para entender qué puede y qué no puede hacer un sistema de inteligencia artificial (Dalkir, 2017). Imagina que trabajas en el área de tecnología de un banco y tienes acceso a los registros de transacciones de los últimos seis meses.

Los datos son los registros en bruto: hora, monto, comercio, ubicación, número de tarjeta. Sin contexto, son solo valores. "23:47", "$4.200", "Buenos Aires", "tarjeta 4521", no dicen nada por sí solos y no hay nada que interpretar todavía. Un sistema que solo almacena y recupera datos no razona, solo busca.

La información aparece cuando esos datos se organizan y se les da contexto. "Esta tarjeta realizó cinco compras en menos de diez minutos en tres países distintos." Ya hay algo que leer, una imagen que se forma porque responde preguntas del tipo ¿qué pasó?, ¿cuándo?, ¿dónde? Pero todavía no se sabe qué hacer con eso. Un sistema que produce información puede ordenar, filtrar y relacionar datos, pero todavía no toma decisiones.

El conocimiento es el paso siguiente y es donde empieza a tener importancia para los sistemas de IA. Es la capacidad de usar esa información para resolver un problema. Saber que ese patrón "múltiples transacciones rápidas en ubicaciones geográficamente imposibles" es un indicador claro

de fraude, y que el protocolo correcto es bloquear la tarjeta y notificar al titular. Eso es conocimiento, información que se puede aplicar.

Dalkir (2017) describe que los datos son hechos sin interpretar, la información es datos con contexto y el conocimiento es la capacidad de actuar a partir de esa información para resolver problemas concretos. La diferencia no es de cantidad, sino de naturaleza, porque se puede tener mucha información y poco conocimiento.

Y la sabiduría, que a veces se omite en presentaciones más técnicas, es saber cuándo una regla tiene excepciones y por qué. Saber que ese mismo patrón de transacciones podría corresponder a un cliente que está de viaje y avisó previamente, o que hay un comercio en particular que por razones técnicas siempre registra ubicaciones incorrectas, o que bloquear la tarjeta de ese cliente en ese momento específico podría causarle un daño desproporcionado.

La sabiduría no aplica el conocimiento mecánicamente, sino que lo pondera, lo contextualiza, lo cuestiona cuando la situación lo requiere. Es el juicio que va más allá de las reglas, y es también lo más difícil de replicar en un sistema artificial. Los sistemas expertos clásicos pueden tener mucho conocimiento codificado y muy poca sabiduría.

Figura 1. Representación de jerarquía DIKW



Fuente: elaboración propia.

¿Por qué importa esta distinción para la IA? Porque muchos sistemas que se llaman inteligentes operan en realidad solo con datos o con información. La diferencia entre un buscador y un sistema experto es exactamente esa, ya que uno recupera información, el otro razona con conocimiento y la diferencia entre un sistema experto y un experto humano tiene mucho que ver con esa última capa, la sabiduría, que todavía no se sabe formalizar bien.

Representación del conocimiento

Una vez que se comprende qué es el conocimiento, la pregunta siguiente es cómo representarlo en un sistema computacional de manera que sea útil. No alcanza con almacenarlo, ya que se debe poder utilizar. Poole y Mackworth (2017) identifican cuatro propiedades que cualquier sistema de representación del conocimiento debería cumplir para ser verdaderamente útil.

- **La primera es la suficiencia representacional.** El sistema tiene que poder expresar todo el conocimiento relevante del dominio. Parece obvio, pero es más difícil de lo que suena porque todo lenguaje de representación tiene límites. La lógica proposicional, por ejemplo, puede representar hechos simples, pero no relaciones generales entre objetos. Si el dominio requiere decir "todos los clientes con más de tres tarjetas activas tienen mayor riesgo de fraude coordinado", la lógica proposicional no alcanza y se necesita lógica de primer orden. Cuando un sistema no puede representar ciertas situaciones, comete errores silenciosos, ya que no dice que se equivocó, simplemente no sabe que hay algo que no puede ver.

- **La segunda es la suficiencia deductiva.** El sistema tiene que poder manipular ese conocimiento para obtener conclusiones nuevas. Esto es lo que separa una base de conocimiento de una base de datos. Si el sistema solo puede devolver lo que fue explícitamente cargado, no razona: consulta. Un sistema con suficiencia deductiva puede tomar hechos que nadie relacionó explícitamente y derivar conclusiones que no estaban almacenadas en ninguna parte.

- **La tercera es la eficiencia deductiva.** Tiene que hacerlo de manera dirigida y razonablemente rápida. No alcanza con que el sistema sea capaz de derivar conclusiones en principio: tiene que poder hacerlo en tiempo útil. Un motor de inferencia que necesita

explorar millones de caminos para llegar a una conclusión simple es inviable en producción. La eficiencia deductiva es el puente entre la corrección teórica y la utilidad práctica, y es uno de los problemas más duros en este campo.

- **La cuarta es la eficiencia en la adquisición.** El sistema tiene que poder incorporar conocimiento nuevo sin que todo colapse. Los dominios cambian. Las reglas de fraude de hace cinco años no son las de hoy. Un sistema que requiere ser reescrito desde cero cada vez que el mundo cambia tiene una vida útil muy corta. Esta propiedad es también la que hace que la ingeniería del conocimiento sea tan costosa: no basta con cargar el conocimiento una vez, hay que mantenerlo actualizado y consistente a lo largo del tiempo.

Cuando alguna de estas cuatro condiciones falla, el sistema tiene problemas. En sistemas que son críticos, como detección de fraude, diagnóstico médico, control de infraestructura, estos problemas tienen consecuencias reales.

Inferencia y razonamiento

La inferencia es el mecanismo por el cual un sistema deriva conclusiones nuevas a partir de lo que ya conoce. Se parte de un conjunto de proposiciones verdaderas llamadas axiomas y se aplican reglas formales para obtener proposiciones nuevas.

Como señalan Poole y Mackworth (2017), es un proceso esencialmente sintáctico. La máquina no necesita comprender el significado de lo que procesa, solo seguir las reglas. Un sistema de detección de fraude no necesita saber qué significa "fraude", le basta con saber que, si se cumplen ciertas condiciones, se aplica cierta regla.

La regla de inferencia más elemental establece que, si sabes que "si P, entonces Q" y además se sabe que P es verdadero, podemos concluir que Q también es verdadero. Formalmente, a partir de las premisas $P \rightarrow Q$ y P, se deriva Q como una conclusión válida.

Parece trivial, pero es el mecanismo que sostiene prácticamente todo sistema experto. Encadenando muchas aplicaciones de esta regla, un sistema puede llegar a conclusiones complejas a partir de hechos simples.

> El razonamiento es algo más amplio. Incluye la inferencia, pero también la selección de qué inferir, cuándo detenerse, cómo priorizar entre varias líneas posibles y cómo manejar información contradictoria o incompleta. El razonamiento humano combina lógica con intuición, reconocimiento de patrones y experiencia acumulada de maneras que todavía no se sabe replicar completamente en un sistema formal.

Vale la pena mencionar una distinción técnica que se retomará más adelante. Se dice que Q es una consecuencia semántica de un conjunto de fórmulas cuando Q es verdadera en todos los modelos en los que esas fórmulas también son verdaderas. En cambio, se dice que Q es derivable sintácticamente cuando existe una secuencia de pasos formales que permite llegar desde las fórmulas iniciales hasta Q.

Un buen sistema de inferencia debe cumplir dos propiedades fundamentales: ser correcto, es decir, derivar únicamente conclusiones válidas, y ser completo, es decir, permitir derivar todas las conclusiones que son consecuencia lógica de las premisas. La lógica de primer orden, que estudiarás en detalle más adelante, cumple ambas propiedades bajo determinadas condiciones. Sin embargo, estas condiciones implican un costo computacional, y es allí donde comienzan a aparecer algunas de las principales dificultades prácticas.

Base de conocimiento

El componente central de cualquier agente basado en conocimiento es la base de conocimiento (BC), que Russell y Norvig (2016) describen como el repositorio donde el agente almacena todo lo que sabe sobre su dominio: los objetos relevantes, sus propiedades, las relaciones entre ellos y las reglas que los gobiernan. Junto a la BC, el agente tiene un motor de inferencia, que es el mecanismo que aplica las reglas para derivar conclusiones y una interfaz que le permite interactuar con el mundo.

La distinción con una base de datos clásica es fundamental, ya que esta última devuelve exactamente lo que alguien puso ahí. Si el dato no fue cargado explícitamente, no existe. Una BC puede derivar información que nunca fue almacenada de manera directa. Si el sistema sabe que toda transacción con ciertas características es sospechosa, y recibe una transacción nueva con esas

características, puede concluir que es sospechosa sin que nadie le haya dicho nada sobre esa transacción en particular.

Esa capacidad de derivación es lo que la hace útil para razonar. Toda base de conocimiento se opera con dos acciones básicas: cargar información nueva en el sistema y consultarle si algo es verdadero.

Para el ejemplo, se le dice al sistema que todos los empleados con más de cinco años de antigüedad tienen derecho a vacaciones extendidas. Se dice también que María lleva siete años en la empresa. Luego se le pregunta si María tiene derecho a vacaciones extendidas. El sistema responde que sí, no porque nadie se lo haya dicho explícitamente sobre María, sino por que lo derivó aplicando la regla al hecho.

Construir una BC implica extraer el conocimiento de expertos humanos (que muchas veces no saben articular exactamente cómo toman sus decisiones), estructurarlo, formalizarlo en un lenguaje que el sistema pueda procesar y verificar que sea consistente.

Ese proceso, conocido como ingeniería del conocimiento, es uno de los más costosos en el desarrollo de sistemas simbólicos y uno de sus principales cuellos de botella históricos. En los años ochenta y noventa, proyectos de sistemas expertos fracasaron no porque la tecnología no funcionara, sino porque adquirir y mantener el conocimiento resultó mucho más difícil y caro de lo esperado. Se volverá sobre esto con más detalle cuando se llegue a los sistemas expertos.

Formas de representar el conocimiento

Hay tres grandes familias para representar conocimiento en un sistema de IA, no son excluyentes, ya que los sistemas reales suelen combinarlas, pero vale conocerlas por separado porque cada una tiene sus fortalezas, sus limitaciones y sus casos de uso naturales (Poole y Mackworth, 2017).

Lógica

Toca para ver detalle

(Información del reverso a continuación)

Lógica

Los hechos y relaciones se expresan en un lenguaje formal con sintaxis y semántica precisas. Lo importante de esta representación es que permite garantizar la corrección del razonamiento, es decir, si las premisas son verdaderas y las reglas se aplican bien, las conclusiones son necesariamente verdaderas. Esto la hace ideal para dominios donde la precisión es fundamental, tales como ontologías médicas que estructuran el conocimiento clínico, los lenguajes de consulta como SQL, los sistemas de verificación formal de software, entre otros.

Tiene sus limitaciones porque es expresiva pero costosa computacionalmente y no maneja bien la incertidumbre. En lógica clásica, una proposición es verdadera o falsa; mientras que el mundo real rara vez es tan ordenado.

Sistemas de reglas

Toca para ver detalle

(Información del reverso a continuación)

Sistemas de reglas

El conocimiento se codifica como pares "SI... ENTONCES", donde la parte SI describe condiciones que deben cumplirse y la parte ENTONCES describe la acción o conclusión que se sigue: "si la transacción es nocturna Y la IP es extranjera Y la cuenta no tiene historial internacional, entonces clasificar como alto riesgo".

Esta representación tiene una ventaja sobre la lógica pura, porque es inmediatamente legible por personas no técnicas. Un analista de fraude puede leer las reglas, entenderlas, cuestionarlas y proponer modificaciones sin necesidad de conocer la lógica formal.

Su limitación principal es la escalabilidad; cuando hay miles de reglas, puede ser difícil garantizar que no sean contradictorias entre sí, y el motor de inferencia puede volverse lento si no está bien diseñado. También son frágiles ante situaciones que nadie anticipó al escribir las reglas.

Objetos estructurados

El conocimiento se organiza alrededor de entidades con atributos y relaciones entre ellas. Una cuenta bancaria es un objeto con atributos (titular, saldo, antigüedad, historial) y relaciones con otros objetos (tarjetas asociadas, sucursal, titulares adicionales). Las redes semánticas representan este conocimiento como grafos donde los nodos son entidades y los arcos son relaciones. Los frames son estructuras más ricas que agrupan todo lo que se sabe sobre un tipo de entidad, incluyendo valores por defecto y procedimientos para calcular atributos que no están almacenados explícitamente.

Esta forma de representación es la que más se parece intuitivamente a cómo los humanos organizan el conocimiento sobre el mundo; en términos de cosas y sus propiedades. Es también el antecedente directo de la programación orientada a objetos y de los grafos de conocimiento que usan empresas como Google o Meta para estructurar información a escala masiva.

- **Redes semánticas**

Una red semántica representa el conocimiento como un grafo. Sus elementos básicos son los nodos, que representan conceptos, objetos o entidades del dominio; los arcos, que representan las relaciones entre esos conceptos (es-un, parte-de, tiene); y los procedimientos de inferencia, que operan sobre la estructura para derivar relaciones implícitas.

Su poder está en la inferencia sobre estructura: si la red tiene el nodo "Cuenta_4521" conectado por "es-una" a "Cuenta de alto riesgo", y "Cuenta de alto riesgo" conectado por "requiere" a "revisión manual", el sistema puede inferir que Cuenta_4521 requiere revisión manual aunque nadie lo haya indicado explícitamente para esa cuenta. El sistema sabe más de lo que le dijeron.

- **Frames**

Los frames son descripciones estructuradas de un objeto o clase de objetos. Cada frame tiene un nombre y un conjunto de ranuras (*slots*) con sus valores, y son la base conceptual de la programación orientada a objetos. Sus ventajas principales son que permiten describir cada concepto con todos sus atributos de forma organizada, que el acceso a los atributos es rápido y que puede heredar atributos de otro más general sin necesidad de repetirlos.

Un frame para "transferencia internacional sospechosa" tendría *slots* como:

- Monto (supera tres veces el promedio histórico del cliente).
- IP de origen (país sin historial previo en la cuenta).
- Documentación adjunta (ausente).

- ✓ Historial del destinatario (sin operaciones previas con esa cuenta).
- ✓ Acción recomendada (bloqueo preventivo y alerta al equipo de cumplimiento).

Cuando el sistema instancia ese frame con los datos de una transferencia específica, tiene una representación estructurada completa lista para comparar contra las reglas vigentes y tomar una decisión.

Las redes priorizan las relaciones entre conceptos y la inferencia que se puede hacer a partir de ellas; los frames priorizan la descripción completa de un objeto con todos sus atributos. En la práctica, los sistemas reales suelen combinar ambas.

Tabla 1. Representación del conocimiento

Forma de representación	Principio	Representación	Ejemplo de uso
Lógica	Hechos y relaciones en lenguaje formal con semántica precisa.	Prolog, Base de datos relacionales.	Verificar si un paciente cumple los criterios de un diagnóstico.
Reglas	Conocimiento como pares condición-acción.	SI/ENTONCES.	Marcar una transacción como riesgosa si cumple ciertas condiciones.

Forma de representación	Principio	Representación	Ejemplo de uso
Objetos estructurados	Entidades con atributos y relaciones entre ellas.	Redes semánticas, frames, grafos de conocimiento.	Representar a un cliente con sus cuentas, tarjetas y sucursal asociada.

Fuente: elaboración propia.

Tipos de razonamiento

Antes de entrar en la lógica formal, conviene nombrar los distintos tipos de razonamiento que existen, ya que ayuda a comprender el alcance real de las herramientas que se verán, sus fortalezas y sus límites.

El razonamiento deductivo va de lo general a lo particular. Si las premisas son verdaderas y la regla se aplica correctamente, la conclusión está garantizada sin margen de error lógico. Siguiendo el ejemplo del banco: "toda transacción con estas cinco características es fraudulenta. Esta transacción las tiene. Por lo tanto, esta transacción es fraudulenta". No hay posibilidad de error dado que las premisas son correctas.

Este es el razonamiento que la lógica formal puede capturar completamente, y sobre el que se construye la mayor parte de lo que se estudiará en esta lectura. Su fortaleza, la certeza, es también su limitación, ya que solo funciona cuando las reglas generales son correctas y completas. En dominios complejos y cambiantes no siempre se puede garantizar.

El razonamiento inductivo va en la dirección opuesta: de casos particulares hacia una generalización: "en las últimas diez mil transacciones fraudulentas detectadas, el 94% ocurrió de noche, involucró montos superiores a cierto umbral y se realizó desde una ubicación distinta a la habitual del cliente". La conclusión, que esos factores son indicadores de fraude, es probable y no garantizada. Puede haber excepciones, y el patrón puede cambiar con el tiempo a medida que los defraudadores adaptan sus métodos. Este es el tipo de razonamiento que subyace al aprendizaje automático; en lugar de que alguien escriba las reglas explícitamente, el sistema las infiere a partir de ejemplos (Murphy, 2022).

El razonamiento abductivo, relevante para la IA aplicada, funciona al revés del deductivo. Parte de una observación y busca la explicación más plausible. Plausible, a modo de recuerdo, significa que algo es creíble o razonable dado lo que se sabe, aunque no esté confirmado ni garantizado. Si el sistema detecta una transacción sospechosa, razona hacia atrás para determinar qué la explica: ¿es fraude?, ¿es el cliente viajando?, ¿es un error del sistema? Los sistemas de diagnóstico (médico, técnico, de seguridad) hacen abducción constantemente, aunque no siempre lo llamen así.

El razonamiento analógico extrae conclusiones por similitud entre casos: "este patrón de movimientos en la cuenta se parece mucho al que se registró seis meses antes de aquel esquema de fraude coordinado entre comercios. Probablemente, estemos ante algo similar". Es muy propio del razonamiento humano experto, especialmente en dominios donde la experiencia acumulada es fundamental, pero difícil de formalizar de manera rigurosa. Los sistemas de razonamiento basado en casos, son el intento más sistemático de implementarlo en IA.

El razonamiento analítico descompone un problema en sus partes para entenderlo. Un analista que, ante un modelo de scoring con muchos falsos positivos, revisa variable por variable, descarta las que no discriminan y encuentra que el problema está en cómo se está calculando la antigüedad del cliente, está usando este tipo de razonamiento. Es el razonamiento del diagnóstico sistemático: divide el espacio de posibilidades y va eliminando hasta encontrar la causa.

El razonamiento sintético hace lo contrario, integra múltiples fuentes de información parcial para construir una imagen completa. Combinar el historial transaccional, la ubicación del dispositivo, el comportamiento reciente y los datos demográficos del cliente para determinar si una operación es legítima es un ejemplo de razonamiento sintético. También lo es construir un perfil de riesgo consolidado a partir de múltiples señales que, por separado, ninguna sería suficiente para tomar una decisión.

Un experto humano usa todos estos tipos simultáneamente, sin pensarlo, pasando de uno a otro según lo que el problema requiere. Un sistema de IA tiene que implementar cada uno de manera explícita y separada. Esa diferencia es importante y explica en parte por qué los sistemas simbólicos puros tienen un techo; son muy buenos en razonamiento deductivo, razonables en analítico y abductivo cuando el espacio de hipótesis está bien definido; pero bastante limitados en inductivo y analógico. Los sistemas de aprendizaje automático, por el contrario, son excelentes en inductivo, pero débiles en deductivo. Esto es lo que motiva el interés en sistemas híbridos que combinen ambos paradigmas.

Figura 2. Tipos de razonamiento



Fuente: elaboración propia.

Lógica proposicional

Toda forma de razonamiento, ya sea humano o artificial, se apoya sobre algún tipo de representación. Para poder razonar, un sistema necesita primero poder hablar sobre el mundo de una manera estructurada y no ambigua. La lógica proposicional es el sistema formal más sencillo y, al mismo tiempo, el fundamento sobre el que se construyen todas las lógicas más expresivas.

Russell y Norvig (2016) la definen como un lenguaje formal en el que las oraciones, llamadas proposiciones o fórmulas, pueden ser verdaderas o falsas, y cuya combinación mediante conectores lógicos permite derivar nuevas verdades a partir de verdades conocidas. La clave está en esa última parte, precisamente derivar. No se trata de memorizar hechos, sino de inferirlos. Un sistema que solo memoriza hechos es una base de datos; un sistema que infiere nuevos hechos a partir de los conocidos es un sistema que razona.

Proposiciones: sintaxis y semántica

Una proposición atómica es una afirmación declarativa que puede evaluarse como verdadera o falsa sin ambigüedad de contexto: "la transacción 8823 supera los diez mil pesos" es una proposición porque tiene un valor de verdad definido. En cambio, "¿es esta transacción fraudulenta?", "bloquea la cuenta" u "ojalá no sea fraude" no son proposiciones en sentido lógico, porque una es una pregunta, otra una orden y la tercera una expresión de deseo; ninguna puede evaluarse como verdadera o falsa.

Murphy (2022) distingue entre la sintaxis y la semántica de un lenguaje lógico como elemento conceptual fundamental. La sintaxis define qué secuencias de símbolos son fórmulas bien formadas; la semántica define qué significan esas fórmulas en relación con el mundo. Un sistema que solo verifica la sintaxis puede detectar fórmulas mal construidas, pero solo la evaluación semántica determina si son verdaderas o falsas en un mundo concreto.

Es importante señalar que la evaluación como verdadera o falsa depende del estado del mundo en un momento dado. La misma proposición "el tráfico supera 10.000 req/s" puede ser verdadera a las 14:30 y falsa a las 14:35. En lógica proposicional clásica, se trabaja con un mundo estático; las extensiones dinámicas abordan mundos que cambian con el tiempo.

Conectores lógicos

Los conectores lógicos permiten construir proposiciones compuestas a partir de proposiciones atómicas, como se muestra en la tabla presentada a continuación.

Para aplicar estos conectores en un contexto real de detección de fraude, se considera el siguiente ejemplo: sea P = "la transacción supera tres veces el monto promedio del cliente" y Q = "se realizó desde una ubicación distinta a la habitual" y R = "la cuenta tuvo tres o más intentos fallidos de autenticación en las últimas dos horas". Entonces, la regla $(P \land Q) \lor R \rightarrow$ BloquearOperación captura la intuición de que debe bloquearse la operación cuando hay indicios combinados de uso anómalo (monto inusual desde una ubicación desconocida) o cuando hay evidencia directa de intento de acceso no autorizado, independientemente del monto o la ubicación. Esta formalización permite que un motor de reglas procese millones de transacciones por segundo de manera consistente y totalmente auditable.

Tabla 2. Conectores lógicos

Conector (símbolo)	Definición semántica	Comentarios
Negación ¬P (NOT)	¬P es V si y solo si P es F. Invierte el valor de verdad.	No confundir con incertidumbre: ¬P no significa 'P es desconocida', sino específicamente que P es falsa.

Conector (símbolo)	Definición semántica	Comentarios
Conjunción P ∧ Q (AND)	P ∧ Q es V si y solo si ambas P y Q son V. Cualquier F hace F al conjunto.	Requiere que AMBAS sean verdaderas. En análisis de alertas de seguridad, es más restrictivo que OR.
Disyunción P ∨ Q (OR)	P ∨ Q es V si al menos una de P, Q es V. Solo F si ambas son F.	Es disyunción inclusiva: P ∨ Q puede ser V aunque ambas sean V. Para exclusiva, usar P ⊕ Q (XOR).
Condicional P → Q (si...entonces)	P → Q es F únicamente cuando P es V y Q es F. En todos los otros casos es V.	Paradoja: cuando P es F, P→Q es V independientemente de Q. 'Si el firewall cae, el sistema es seguro' es lógicamente verdadera si el firewall nunca cae.
Bicondicional P ↔ Q (si y solo si)	P ↔ Q es V cuando P y Q tienen el mismo valor de verdad (ambas V o ambas F).	Equivale a (P→Q) ∧ (Q→P). Más restrictivo que el condicional simple.
Negación ¬P (NOT)	¬P es V si y solo si P es F. Invierte el valor de verdad.	No confundir con incertidumbre: ¬P no significa 'P es desconocida', sino específicamente que P es falsa.

Fuente: elaboración propia.

Tablas de verdad

Las tablas de verdad son el mecanismo para evaluar el valor de una fórmula proposicional para todas las posibles combinaciones de valores de sus proposiciones atómicas. Para una fórmula con

n variables proposicionales, la tabla tendrá 2^n filas. A continuación, se construirá una para la fórmula $(P \wedge Q) \rightarrow R$, que aparece en sistemas de diagnóstico condicional.

Tabla 3. Tablas de verdad

P (fiebre alta)	Q (tos)	P ∧ Q (ambas)	(P ∧ Q) → R (requiere evaluación)
V	V	V	V solo si R=V; F si R=F.
V	F	F	V (la premisa es falsa, la implicación es verdadera).
F	V	F	V (ídem).
F	F	F	V (ídem).

Fuente: elaboración propia.

La lógica proposicional tiene una propiedad valiosa, es decidible. Esto significa que siempre existe un procedimiento algorítmico para determinar si una fórmula es verdadera en todos los casos posibles, falsa en todos o verdadera solo en algunos. No hay fórmulas sobre las que el sistema no pueda pronunciarse. Esa certeza se pierde en lógicas más expresivas, como se estudiará más adelante (Murphy, 2022).

El costo de esa decidibilidad es computacional. Determinar si existe alguna combinación de valores de verdad que satisfaga una fórmula, problema conocido como SAT, crece exponencialmente con la cantidad de variables. Para fórmulas pequeñas alcanza con construir una tabla de verdad, para fórmulas con decenas o cientos de variables, ese enfoque se vuelve inviable. En la práctica se usan algoritmos especializados que resuelven el problema de manera mucho más eficiente, aunque sin garantía de rapidez en el peor caso.

Reglas de inferencia

Hasta ahora se habló de representar conocimiento, las reglas de inferencia son el mecanismo que permite derivar sentencias nuevas a partir de las que ya se conocen, con la garantía de que, si las premisas son verdaderas, la conclusión también lo es. Russell y Norvig (2016) las presentan como el mecanismo que transforma un conjunto estático de hechos en un sistema capaz de responder preguntas que nadie almacenó explícitamente.

> El modus ponens es la regla más usada y la idea detrás es simple: de P y P→Q se concluye Q. Cuando una cosa lleva a otra y se confirma que la primera ocurrió, la segunda también ocurrió.

En un sistema bancario, P es "el cliente retiró más del ochenta por ciento de su saldo en efectivo dentro de las veinticuatro horas posteriores a recibir una transferencia de origen desconocido". P→Q es la regla que establece que ese comportamiento implica marcar la operación para revisión. Cuando el sistema detecta que P se cumple en una cuenta concreta, concluye Q sin que nadie se lo haya dicho explícitamente sobre esa cuenta. Lo derivó aplicando la regla al hecho. Es la base del encadenamiento hacia adelante y el mecanismo central de los sistemas expertos.

> El modus tollens funciona en la dirección contraria: de ¬Q y P→Q se concluye ¬P. Si una regla establece que cierta condición siempre produce cierto resultado, y ese resultado no aparece, entonces la condición tampoco se cumplió.

En el mismo contexto, P→Q es la regla que dice que las transferencias al exterior de una empresa importadora siempre van acompañadas de documentación adjunta en el sistema. ¬Q es la observación de que llegó una transferencia internacional de esa cuenta sin ningún documento asociado. El sistema concluye ¬P: esa operación no responde al comportamiento habitual del cliente. No confirmó fraude; descartó legitimidad: esa diferencia es central en el diagnóstico diferencial: en lugar de buscar qué es la operación, el sistema elimina lo que no puede ser.

El silogismo hipotético permite encadenar implicaciones: de P→Q y Q→R se concluye P→R. No hace falta evaluar cada eslabón por separado cada vez; si la cadena está construida, se aplica completa desde el primer disparo.

En la práctica bancaria, P→Q es la regla que dice que superar el umbral de riesgo implica revisión manual. Q→R es la regla que dice que toda operación en revisión manual implica bloqueo preventivo de la cuenta. El sistema infiere directamente P→R: superar el umbral de riesgo implica bloqueo preventivo, sin necesidad de pasar explícitamente por el paso intermedio cada vez. Es el mecanismo fundamental en sistemas de reglas de múltiples etapas.

La resolución es la más general de las cuatro y la base de demostradores automáticos de teoremas. Combina cláusulas para eliminar literales complementarios: si una cláusula contiene P y otra contiene ¬P, ambas pueden combinarse para producir una nueva cláusula que no contiene ninguna de las dos.

En un sistema de análisis de transferencias, el motor parte de que una operación o es legítima o es sospechosa. Tiene evidencia de que no es legítima: el monto supera el promedio histórico del cliente, el destinatario nunca antes había recibido fondos de esa cuenta y el horario es atípico para el perfil. Al eliminar la cláusula de legitimidad, concluye que la transferencia es sospechosa. No llegó a esa conclusión de forma directa, sino por eliminación sistemática de la alternativa contraria.

Lo que hace útiles a estas reglas es que se pueden sistematizar: un analista humano aplica razonamientos similares de manera intuitiva; un motor de inferencia los aplica de manera consistente sobre millones de transacciones por segundo y con trazabilidad completa.

Experimentación: razonamiento lógico formal y sus límites

Plataforma: Registrar cuál IAG se utiliza.

Prompt 1 — Construcción de tablas: tengo las siguientes proposiciones: P = 'el paciente tiene fiebre alta', Q = 'el paciente tiene tos seca', R = 'el paciente tiene dificultad respiratoria'. Construye la tabla de verdad completa para la fórmula $(P \land Q) \rightarrow R$ y explica qué situación clínica representa cada fila.

Prompt 2 — Aplicación a tu dominio: adapta el ejercicio anterior a [tu carrera]. Define 3 proposiciones relevantes y construye una fórmula que represente una regla de decisión de tu dominio. Evalúa su tabla de verdad.

Prompt 3 — Límites: ¿En qué situaciones la lógica proposicional sería insuficiente para representar el conocimiento de tu dominio? ¿Qué tipo de proposiciones necesitarías que no puede expresar?

Reflexión: ¿Identificas casos en que la IA genera razonamientos lógicamente correctos pero semánticamente incorrectos para tu dominio? ¿Qué dice esto sobre la diferencia entre corrección formal y validez contextual?

Lógica de primer orden

La lógica proposicional, a pesar de sus propiedades computacionales favorables, tiene una limitación estructural de fondo, ya que trata el mundo como un conjunto de hechos atómicos sin estructura interna. No puede hablar sobre objetos individuales, sus propiedades, ni las relaciones entre ellos de manera general, no puede expresar "todos los robots con batería inferior al 20% deben regresar a la base de carga" sin crear una proposición separada para cada robot del sistema y no puede capturar "existe al menos un gen en el cromosoma 17 con una mutación que afecta al dominio" sin enumerar cada gen posible. Cuando el dominio involucra objetos, propiedades y relaciones estructuradas, se necesita de la lógica de primer orden (LPO).

Russell y Norvig (2016) la definen como el lenguaje estándar de representación del conocimiento en IA simbólica, señalando que combina la precisión formal de la lógica proposicional con la capacidad de hablar sobre objetos, sus propiedades y las relaciones que mantienen entre sí. Esta combinación la convierte en una herramienta extraordinariamente expresiva, capaz de representar prácticamente cualquier conocimiento de dominio con el nivel de detalle que sea necesario.

Bloques fundamentales

La LPO introduce nuevos elementos sintácticos que no existían en la lógica proposicional. Estos elementos permiten hablar sobre el mundo con la granularidad necesaria para capturar conocimiento real.

Términos: objetos del dominio

Son las expresiones que especifican objetos del mundo. Pueden ser:

- **Constantes:** nombres de objetos específicos. En robótica: Robot_A, Estación_Carga_1. En seguridad: IP_192_168_1_1, Usuario_admin. Las constantes son rígidas porque siempre denotan el mismo objeto en cualquier modelo.

- **Variables:** símbolos que pueden referirse a cualquier objeto del dominio. Se denotan típicamente con letras minúsculas (x, y, z). Su valor se fija mediante cuantificadores o sustitución.

- **Funciones:** mapeos de objetos a objetos. BateriaActual(robot) retorna un valor numérico para cada robot. PadreDe(gen) puede retornar el gen del que deriva una variante. Las funciones permiten construir términos complejos.

Predicados: propiedades y relaciones

Los predicados son la base de la LPO. Un predicado n-ario toma n argumentos (términos) y se evalúa como verdadero o falso para cada combinación de argumentos en un modelo dado. Un predicado unario expresa una propiedad de un objeto, un predicado binario expresa una relación entre dos objetos y así sucesivamente.

Tabla 4. Ejemplos de predicados

Predicado	Aridad	Ejemplo de uso en LPO
EstaOperativo(robot)	Unario.	EstaOperativo(Robot_A): verdadero si Robot_A está en condiciones de operar.
TieneObstaculo(robot, zona)	Binario.	TieneObstaculo(Robot_A, pasillo_3): verdadero si Robot_A detecta un obstáculo en pasillo_3.
NivelBateria(robot, nivel)	Binario.	NivelBateria(Robot_A, critico): verdadero si Robot_A tiene batería en nivel crítico.
AsignadoA(robot, tarea, zona)	Ternario.	AsignadoA(Robot_A, inspeccion, sector_B): verdadero si Robot_A tiene asignada una inspección en sector_B.

Fuente: elaboración propia.

Los predicados son distintos de las funciones. Una función retorna un objeto del dominio, un predicado retorna un valor de verdad. Esta distinción es la diferencia entre "la temperatura actual del servidor" (una función que retorna un número) y "el servidor está sobrecalentado" (un predicado que es verdadero o falso).

Cuantificadores: universalidad y existencia

Los cuantificadores son el mecanismo mediante el cual la LPO logra hacer afirmaciones sobre conjuntos de objetos sin enumerarlos explícitamente. Son, en muchos sentidos, la diferencia fundamental entre la lógica proposicional y la de primer orden.

Cuantificador universal (∀)

El cuantificador universal (∀, "para todo") permite expresar afirmaciones que aplican a todos los objetos del dominio que satisfacen cierta condición. Murphy (2022) destaca que este cuantificador es esencial para expresar leyes, regularidades y restricciones generales del dominio, que son precisamente el tipo de conocimiento que los expertos humanos articulan cuando describen su campo. Se considera una regla de detección de fraude bancario:

```
∀t [ Transaccion(t) ∧ MontoSuperaUmbral(t, 50000) ∧ OrigenIPExtranjera(t) ∧
¬DocumentacionAdjunta(t) ] → RiesgoAlto(t)
```

Esta regla detecta transacciones que combinan un monto elevado, origen geográfico inusual y ausencia de documentación respaldatoria, un patrón típico de operaciones sospechosas que los analistas de cumplimiento reconocen como señal de alerta. El cuantificador universal permite que la regla aplique a cualquier transacción presente o futura, sin necesidad de examinar cada caso manualmente ni enumerar situaciones particulares.

Cuantificador existencial (∃)

El cuantificador existencial (∃, "existe") afirma que al menos un objeto del dominio satisface una propiedad. Es menos restrictivo que el universal porque no requiere que todos los objetos satisfagan la condición, solo que alguno lo haga.

```
∃t [ Transaccion(t) ∧ Involucra(t, cuenta_cliente) ∧ RealizadaEnExterior(t)
∧ SuperaMonto(t, 100000)]
```

Esta fórmula afirma que existe al menos una transacción internacional de alto monto asociada a la cuenta del cliente. Es la formalización de un hallazgo de perfil y no especifica cuál transacción es ni cuándo ocurrió, solo confirma que hay alguna. Esto es fundamental en el análisis de cumplimiento normativo (compliance), donde a menudo lo que importa en un primer paso es saber si un cliente presenta determinado tipo de operatoria, antes de profundizar en los detalles de cada movimiento.

Russell y Norvig (2016) destacan que los cuantificadores pueden anidarse y combinarse para expresar afirmaciones de gran complejidad. Por ejemplo, para afirmar que cada cuenta de alto riesgo registra al menos una transacción sin documentación respaldatoria en el último mes:

```
∀c [ Cuenta(c) ∧ PerfilAltoRiesgo(c) ] → ∃t [ Transaccion(t) ∧ Involucra(t, c) ∧ UltimoMes(t) ∧ ¬DocumentacionAdjunta(t) ]
```

Este tipo de anidamiento permite capturar restricciones de estructura compleja que serían imposibles de expresar en lógica proposicional. En el contexto bancario, esta fórmula podría activar una auditoría automática sobre todas las cuentas categorizadas como alto riesgo que no estén cumpliendo con los requisitos documentales de la regulación vigente.

Sistemas basados en reglas

Una vez que se comprenda cómo representar conocimiento, el paso siguiente es construir sistemas que realmente lo usen. Hasta ahora se describieron hechos, reglas e inferencia. Los sistemas basados en reglas (SBR) son la forma más directa de juntar todo eso en una arquitectura que almacena conocimiento en forma de reglas IF-THEN y las aplica sistemáticamente sobre una situación dada para derivar conclusiones o ejecutar acciones.

Históricamente, son la primera forma de IA que tuvo impacto real y demostrable en el mundo: en diagnóstico médico, en gestión de crédito, en control industrial. Y aunque hoy hay mucho entusiasmo por el aprendizaje automático, los SBR siguen siendo relevantes en cualquier contexto donde la transparencia y la trazabilidad de las decisiones no son opcionales. En banca, seguros, salud y aviación, cuando un sistema toma una decisión que afecta a una persona, alguien tiene que poder explicar por qué. Un sistema de reglas puede hacer eso, mientras que una red neuronal normalmente no.

¿Cómo se representa el conocimiento en un SBR?

Antes de entrar en la estructura de una regla individual, conviene detenerse en cómo organiza el conocimiento un sistema de este tipo. Porque no todo el conocimiento que necesita un SBR para

funcionar tiene la misma naturaleza, ni cambia a la misma velocidad, ni puede ser modificado por las mismas personas.

En un SBR, el conocimiento se distribuye en dos capas:

- **La primera es la base de reglas,** que contiene el conocimiento permanente y general del dominio: lo que siempre es verdad, las políticas que rigen las decisiones, las relaciones entre condiciones y consecuencias. En un banco, esto incluye las reglas que definen qué patrones son sospechosos, qué umbrales disparan una alerta, qué acciones corresponden a cada nivel de riesgo. Este conocimiento lo construyen y mantienen expertos del dominio, analistas de cumplimiento, equipos de prevención de fraude. No cambia con cada transacción, cambia cuando cambian las regulaciones, cuando se detectan nuevos patrones de fraude, cuando la institución revisa sus políticas.

- **La segunda capa es la memoria de trabajo,** que contiene el conocimiento situacional y transitorio: los hechos concretos del caso que el sistema está analizando en este momento. Para una transacción específica, eso es el monto, el horario, la IP de origen, el historial reciente del cliente, si hay o no documentación adjunta. Esta capa la alimenta el mundo externo en tiempo real y su contenido es completamente distinto para cada caso.

Esa separación tiene una consecuencia práctica, ya que si hay que actualizar una política porque el regulador cambió los umbrales de reporte, se modifica la base de reglas y el sistema inmediatamente aplica los nuevos criterios a todas las transacciones siguientes, sin tocar nada más. Si llega una transacción nueva, se carga en la memoria de trabajo y el sistema la evalúa contra las reglas vigentes, sin necesidad de recompilar ni reconfigurar nada.

Dentro de la base de reglas conviven tres tipos de conocimiento con roles distintos. El conocimiento declarativo describe qué es verdad sobre el dominio, el conocimiento procedimental describe qué hacer ante una situación determinada, el conocimiento de control describe cómo decidir cuando varias reglas aplican al mismo tiempo.

Este último es el que menos se menciona en las introducciones a los SBR y el que más problemas causa en la práctica. Una base de reglas sin conocimiento de control explícito es una base de reglas con comportamiento impredecible cuando el mundo se vuelve complejo, que es exactamente

cuando más se necesita que el sistema sea confiable. Se volverá sobre esto cuando se estudie el motor de inferencia.

Tabla 5. Tipos de conocimiento en un SBR

Tipo de conocimiento	Qué describe	Ejemplo bancario	Quién lo define
Declarativo	Qué es verdad sobre el dominio.	Una operación es de alto riesgo si supera tres veces el promedio del cliente.	Analistas del dominio.
Procedimental	Qué hacer ante una situación.	Si el riesgo es alto, bloquear y alertar al equipo de cumplimiento.	Equipos de operaciones y cumplimiento.
De control	Cómo priorizar entre reglas que compiten.	Las reglas de fraude tienen prioridad sobre las de scoring crediticio.	Arquitectos del sistema junto con expertos.

Fuente: elaboración propia.

Esta distinción no es solo teórica, cuando en los años ochenta y noventa muchos proyectos de sistemas expertos fracasaron, no fue porque la tecnología no funcionara. Fue porque nadie había pensado bien cómo separar estos tres tipos de conocimiento, cómo mantenerlos actualizados de forma independiente y quién era responsable de cada uno. Una base de reglas donde el conocimiento declarativo, el procedimental y el de control están mezclados en las mismas reglas se vuelve imposible de mantener a medida que crece.

Reglas de producción

Una regla IF-THEN tiene dos partes, una condición (SI) que puede combinar varios criterios con operadores lógicos y una conclusión (ENTONCES) que define qué ocurre cuando esos criterios se

cumplen. Lo que hace útil a una regla en la práctica es la información que la acompaña:

- **Identificador único:** permite referenciarla y rastrearla sin ambigüedad.
- **Prioridad:** define qué regla gana cuando dos entran en conflicto.
- **Factor de confianza:** indica qué tan confiable es la conclusión.
- **Metadatos:** versión, autor, fecha.

A continuación, se retomará el contexto bancario que utilizó anteriormente. La siguiente regla detecta un patrón típico de operaciones sospechosas que los analistas de cumplimiento reconocen como señal de alerta: una transacción que combina un monto inusual para el cliente, un origen geográfico distinto al habitual y ausencia de documentación respaldatoria.

Tabla 6. Regla para detectar un patrón de operaciones sospechosas

Componente	Contenido
Nombre	REGLA_FRAUDE_TRANSF_001.
Condición 1	El monto de la transferencia supera tres veces el promedio histórico del cliente.
Condición 2	La IP de origen pertenece a un país sin historial previo en la cuenta.
Condición 3	No hay documentación adjunta en el sistema para esta operación.
Operador	AND (todas deben cumplirse simultáneamente).

Componente	Contenido
Acción 1	Marcar la operación para revisión manual en el equipo de cumplimiento.
Acción 2	Bloquear preventivamente la transferencia hasta resolución.
Acción 3	Generar alerta con contexto completo: monto, origen, historial del cliente.
Prioridad/Confianza	ALTA / 0,91.
Metadatos	Equipo Prevención de Fraude. Versión 3.1. Marzo 2024.

Fuente: elaboración propia.

Lo importante no es solo que las tres condiciones sean relevantes por separado, sino que juntas forman una señal muy difícil de producir en una operación legítima. Un cliente habitual que hace una transferencia grande al exterior normalmente tiene documentación, tiene historial de operaciones internacionales o al menos la IP coincide con su perfil. Cuando los tres indicadores se acumulan al mismo tiempo, la probabilidad de anomalía es alta. Esa especificidad es lo que justifica el factor de confianza de 0,91.

El mismo principio IF-THEN se aplica de manera diferente según el dominio. Lo que varía es el tipo de consecuente: en banca se bloquea o alerta, en bioinformática se clasifica una variante, en robótica se interrumpe un movimiento, en análisis de datos se toma una decisión de preprocesamiento, en producto digital se activa una intervención de UX. La lógica estructural es la misma; lo que cambia es a qué sirve.

Memoria de trabajo

La memoria de trabajo es el repositorio de hechos que el sistema considera verdaderos en este momento. No el conocimiento general y permanente (eso está en la base de reglas), sino el estado

específico y actual del mundo.

La base de reglas de un sistema bancario de detección de fraude contiene las políticas de análisis de riesgo, que cambian poco y rara vez. La memoria de trabajo contiene los datos de la transacción que acaba de ingresar: monto, IP, documentación adjunta e historial reciente del cliente. El motor de inferencia evalúa las reglas contra ese estado actual para decidir qué hacer.

La eficiencia de un SBR depende en buena medida de cómo se gestiona este componente. Si el motor de inferencia necesita evaluar cada regla contra cada hecho cada vez que algo cambia, el sistema no escala. Un banco procesa millones de transacciones diarias; evaluar cada operación contra una base de cientos de reglas en tiempo real requiere que el motor sea inteligente sobre qué reevalúa y qué no. Hay algoritmos que construyen una red de nodos que propagan los cambios de manera incremental, evitando reevaluaciones innecesarias; clave en la historia de la IA simbólica.

Motor de inferencia

El motor de inferencia es el componente que conecta la base de reglas con la memoria de trabajo. Su función es decidir qué reglas aplicar, en qué orden, y cómo proceder cuando varias reglas son válidas al mismo tiempo. Para eso existen dos estrategias principales, cada una más adecuada para cierto tipo de problemas.

Encadenamiento hacia adelante

Esta estrategia parte de los hechos disponibles y los usa para derivar conclusiones nuevas, que a su vez pueden generar más conclusiones y así sucesivamente hasta que el sistema llega a un resultado o agota las reglas aplicables. Es un razonamiento que avanza desde lo que se sabe hacia lo que se puede concluir.

El ciclo funciona en tres pasos:

1. Primero se identifican todas las reglas cuyas condiciones están satisfechas por los hechos actuales.

2. Luego se selecciona una regla según algún criterio de prioridad, se ejecuta su conclusión y el proceso se repite.

3. Cuando muchas reglas se aplican al mismo tiempo, el orden en que se ejecutan puede cambiar el resultado.

Las estrategias más comunes son darle prioridad a la regla más específica, a la que involucra los hechos más recientes o a la que el diseñador marcó explícitamente como prioritaria.

Encadenamiento hacia atrás

Esta estrategia invierte la lógica: en lugar de partir de los hechos y ver a dónde llevan, parte de una hipótesis que se quiere verificar y trabaja en sentido inverso para determinar qué hechos serían necesarios para confirmarla. Como solo explora lo que es relevante para ese objetivo, resulta especialmente eficiente cuando el universo de hechos posibles es enorme, pero los objetivos a verificar son pocos.

El mecanismo es recursivo, ya que para probar una hipótesis, el sistema busca las reglas que podrían establecerla y luego intenta probar las condiciones de cada una, que se convierten en nuevos subobjetivos:

- ✓ Si un subobjetivo ya está en la memoria de trabajo, está resuelto.
- ✓ Si no, el proceso se repite.
- ✓ Si en algún punto no hay forma de avanzar, esa rama se descarta y se prueba otra.

Aplicación al caso

Los conceptos desarrollados a lo largo de esta lectura permiten responder a la pregunta que el caso de MYCIN dejó abierta: ¿qué necesita un sistema para razonar sobre un tema de manera útil, explicable y duradera?

El sistema recibía datos sueltos como temperatura, presión arterial, resultados de análisis e historial de medicamentos. Esos datos solos no decían nada. La información aparecía al organizarlos: "el paciente tiene fiebre y muchos glóbulos blancos", pero el conocimiento aparecía cuando esa combinación, en un paciente internado con catéter, sugería con alta probabilidad una

infección y activaba un protocolo específico. Lo que MYCIN no tenía, como señalaron los médicos que lo evaluaron, era la capacidad de reconocer que la regla general tiene excepciones que dependen del contexto y la experiencia acumulada.

El caso refleja las cuatro propiedades de Poole y Mackworth (2017), ya que MYCIN podía representar razonablemente bien el dominio acotado de las infecciones bacterianas para el que fue diseñado; pero cuando aparecía una infección fuera de ese dominio, el sistema simplemente no tenía cómo explicarla.

Por otro lado, podía llegar a recomendaciones que ningún médico le había indicado de forma explícita. Y era eficiente porque, en lugar de revisar las 600 reglas contra todos los datos del paciente, partía de una hipótesis y solo buscaba la evidencia relevante para confirmarla o descartarla. Sin embargo, su fracaso fue que actualizarlo requería volver a entrevistar a los expertos, un proceso tan costoso que ninguna institución podía sostenerlo con el tiempo.

En un sistema de diagnóstico, la información disponible sobre un paciente es enorme, pero el número de diagnósticos posibles a verificar es acotado. Razonar desde la hipótesis es eficiente exactamente en ese escenario: el sistema trabaja en la dirección que el problema requiere, sin perder tiempo explorando datos que no tienen nada que ver con lo que se está analizando. Si MYCIN hubiera razonado al revés, desde los datos hacia las conclusiones, habría generado una cantidad enorme de conclusiones, la mayoría irrelevantes para lo que el médico necesitaba.

La forma en que MYCIN representaba el conocimiento también muestra los límites de trabajar solo con casos particulares. Las primeras versiones de las reglas tendían a codificar situaciones específicas: "si el paciente Juan García tiene fiebre alta...". La generalización que hace útil a un sistema, "si cualquier paciente internado con estas características tiene estas condiciones...", requiere un nivel mayor de expresión, con reglas que apliquen a cualquier paciente presente o futuro sin necesidad de enumerarlos uno por uno. La diferencia entre ambas formas de escribir las reglas define un salto importante en lo que el sistema puede o no puede hacer.

Finalmente, el caso de MYCIN anticipa una tensión que se explorará en la próxima lectura: la diferencia entre razonar con certeza y razonar con incertidumbre. El razonamiento clásico asume que algo es verdadero o falso, pero en diagnóstico médico, el mejor enunciado posible raramente es "este paciente tiene tal infección", sino "hay una probabilidad de 0,85 de que este paciente tenga tal infección". Los puntajes de certeza de MYCIN fueron el primer intento sistemático de

incorporar esa gradualidad al razonamiento, algo que también se profundizará en la próxima lectura.

Actividades de repaso de lecturas

ACTIVIDADES DE REPASO Y REFUERZO

PREGUNTAS VERDADERO/FALSO

Un sistema de reglas que detecta automáticamente si un paciente cumple los criterios de infección a partir de sus signos vitales opera en el nivel de conocimiento de la jerarquía DIKW.

- ◯ Es verdadero, porque el sistema transforma datos de signos vitales en una conclusión clínica accionable.
- ◯ Es falso, porque un sistema de reglas solo manipula información organizada y el conocimiento requiere comprensión del significado, algo que los sistemas computacionales no pueden tener.

Justificación:

Es verdadero. En el marco DIKW, el conocimiento es la capacidad de aplicar información para resolver problemas concretos. Un sistema que mapea signos vitales a una decisión clínica opera exactamente en ese nivel, independientemente de si el sistema "comprende" el significado en sentido filosófico.

Referencias

Dalkir, K. (2017). *Knowledge management in theory and practice* (3.ª ed.). MIT Press.

Murphy, K. P. (2022). *Probabilistic machine learning: An introduction*. MIT Press.

Poole, D., y Mackworth, A. (2017). *Artificial intelligence: Foundations of computational agents* (2.ª ed.). Cambridge University Press.

Russell, S., y Norvig, P. (2016). *Artificial intelligence: A modern approach* (3.ª ed.). Pearson.

Shortliffe, E. H. (1976). *Computer-based medical consultations: MYCIN*. Elsevier.